Hybrid Ant-Colony Metaheuristics for QoS-Aware Traffic-Engineering Optimization in Software-Defined Networks: Formulation, Exact Certification, Statistical Evaluation, and SRv6 Mapping

João Paulo Versiani Ladeia, and Nilton Alves Maia

***Abstract*—The growth of Internet traffic makes capacity expansion a recurring capital expense and motivates traffic engineering (TE) as a software-based alternative. Software-Defined Networking (SDN) exposes the routing layer to programmatic control, enabling QoS-aware path computation. This paper formulates simultaneous routing of heterogeneous application flows as a constrained combinatorial optimization problem and solves it with a two-stage hybrid: an Ant Colony Optimization stage (AntNet) builds a pool of loop-free candidate routes, and a Genetic Algorithm (ACO+GA) or Differential Evolution (ACO+DE) selects the route set. Beyond the original scoring function, we introduce a revised formulation in which capacity and end-to-end delay are *hard* constraints and load balancing is an explicit min–max link utilization objective, and we solve it exactly as a binary multi-commodity flow (ILP). On a 52-node network and two synthetic 52-node topologies, over five load factors and twenty randomized replications, the exact ILP certifies that the pool-augmented hybrid attains the optimum of its search space in every replication and, more importantly, computes *exact bottleneck lower bounds* — proving, e.g., that at $1.25\times$ load on the sparse instance no routing can keep every link below 98.4% utilization, where shortest-path routing reaches $2.10\times$ capacity and inflicts 6.0 SLA violations against zero for the hybrid. Packet-level simulation shows the optimized network sustaining URLLC-grade flows within a 150 ms delay budget up to $2\times$ background amplification, whereas shortest-path routing violates the same budget at every tested load. A measured data-plane study shows that mapping the optimized paths to SRv6 segment lists reduces forwarding state by $4.1\times$ (from 69 per-flow OpenFlow rules to 17 headend rules) relative to per-flow OpenFlow programming. The results position deterministic, constraint-explicit metaheuristics — with exact certification — as a practical and auditable traffic-engineering core for SDN controllers, and delimit precisely when they are preferable to learning-based routing.**

***Index Terms*—Software-defined networking, traffic engineering, QoS routing, ant colony optimization, genetic algorithm, differential evolution, exact optimization, SRv6, network slicing, deep reinforcement learning.**

I. Introduction

OPERATORS face a structural tension: traffic grows faster than revenue, and capacity expansion is a recurring capital expense. Traffic engineering (TE) attacks this tension from the software side — it routes the traffic that exists more intelligently instead of adding links. Software-Defined Networking (SDN) supplies the missing control point: by decoupling the control plane and exposing programmable forwarding [1]–[3], an SDN controller can compute and install paths that satisfy per-application requirements instead of replicating shortest-path behavior.

This paper studies exactly that problem: given a topology with per-link capacity and delay, and a set of L concurrent applications with minimum bandwidth, maximum delay and priority requirements, compute a route for every application that maximizes admission quality under QoS constraints. We make the following contributions, each of which responds to a concrete methodological gap in the literature on heuristic SDN-TE:

1) **Revised formulation.** We analyze the scoring-function model commonly used in heuristic SDN-TE, identify its degeneracies (constant terms under full admission; capacity counted against a flow cardinality), and reformulate the problem with *hard* capacity and delay constraints and an explicit min–max utilization balancing objective (Section III).
2) **Exact certification.** We solve the revised model exactly as a binary multi-commodity flow ILP and the scoring model exactly over the candidate pool. The exact solutions (i) certify that the pool-augmented hybrid attains the pool optimum in every replication, and (ii) compute *exact bottleneck lower bounds* — proofs of the minimum achievable maximum link utilization — a property no heuristic can claim, which exposes how far greedy and learning-based baselines sit from the provable optimum (Section VIII-A).
3) **Statistical evaluation at scale.** We evaluate six routing methods on three topologies across five load factors and twenty randomized replications, with 95% confidence intervals and Wilcoxon signed-rank tests, identifying the load regime in which the hybrid strictly dominates greedy and learning-based baselines (Section VIII-B).
4) **Measured SRv6 state reduction.** We map the optimized paths to SRv6 segment lists [10]–[12] and *measure* the data-plane state on all topologies, quantifying the

Manuscript received XXX; revised XXX. The authors are with the Graduate Program in Computational Modeling and Systems (PPGMCS), Universidade Estadual de Montes Claros, Montes Claros, Brazil.

reduction with respect to per-flow OpenFlow rules and bounding the SRH overhead (Section V).

5) **Slice isolation evidence.** We connect the demand profile to 3GPP service classes [15], [16] and measure, in packet-level simulation, the stability of URLLC-grade flows under growing background load — comparing optimized against shortest-path routing (Section VI).
6) **Learning-based baseline.** We implement a tabular Q-learning routing agent with the same state–action structure as DRL routers and benchmark it under the same protocol, grounding the DRL discussion in measured numbers rather than assertions (Section VIII-G).

The remainder is organized as follows. Section II reviews related work. Section III presents the system model. Section IV describes the algorithms. Sections V and VI present the SRv6 mapping and slice differentiation. Section VII details the experimental methodology, Section VIII reports results, Section IX discusses threats to validity, and Section X concludes.

II. Related Work

Heuristic and hybrid routing. Ant-based routing originates with AntNet [5], and ACO has been applied to QoS routing repeatedly since. Two-stage "generate-and-select" hybrids — a constructive stage feeding an evolutionary selector — are the standard memetic pattern; the present work instantiates it for SDN with an explicit admission-scoring objective and, critically, adds exact certification that prior hybrid-routing studies omit.

Learning-based routing. DRL for SDN routing has progressed from experience-driven proposals [7], [8] to evaluated traffic-engineering agents [9]. Reported strengths are online adaptivity and traffic-matrix generalization; reported costs are large training campaigns and the sim-to-real gap. Our experiments (Section VIII-G) add a controlled, constraint-explicit comparison on identical instances.

Programmable data planes. OpenFlow [1] programs switches with per-flow rules; P4 [4] generalizes parsing and matching. Both expose the TCAM-pressure problem that motivates SRv6 [10]–[13]: the path state is carried in the packet, and only headends hold policy state. To our knowledge, the state reduction of routing-optimizer output under SRv6 has not been measured on fixed topologies with optimizer-produced paths; Section V reports it.

Network slicing. 3GPP defines slice/service types and QoS flows [16] anchored on the IMT-2020 usage scenarios [15]. Transport-side slice isolation is usually pursued with dedicated resources; we show that constraint-explicit routing yields measurable logical isolation without physical partitioning (Section VI).

III. System Model and Problem Formulation

The network is a directed graph $G = (V, E)$, $|V| = n$; link $(i, j) \in E$ has capacity μ_{ij} (Mbps) and one-way delay c_{ij} (ms). A set of $L = 17$ applications must be routed concurrently; application k requires nominal flow λ_k (Mbps), tolerates end-to-end delay h_k (ms) and carries priority δ_k. Let $x_{ij}^k \in \{0, 1\}$ indicate whether the route of application k uses link (i, j).

A. Original Scoring Model

The original model maximizes the weighted admission score

$$\mathrm{FO} \;=\; \sum_{k=1}^{L} \delta_k\, y_k + p_1 \sum_{k=1}^{L} y_k + p_2 \sum_{k=1}^{L} w_k + p_3 \sum_{k=1}^{L} t_k, \quad (1)$$

with p_1=3, p_2=2, p_3=1; y_k marks an established connection, w_k capacity-sufficient routes and t_k delay-conforming routes. Two structural observations motivate the revised model: (i) under full admission every $y_k = 1$, so the first two sums are constant and the score reduces to $\mathrm{FO} = \Sigma\delta_k + 4L - 2\,n_{\mathrm{over}} - n_{\mathrm{dv}}$, where n_{over} counts overloaded *links* and n_{dv} counts delay-violating *flows*; (ii) the mixed cardinality (links vs. flows) makes weights comparable only by accident. The score saturates at $\mathrm{FO}^* = \Sigma\delta_k + 4L$: any fully feasible solution attains it, and the score cannot discriminate among solutions with different load balance.

B. Revised Model: Hard Constraints and Balancing

We therefore evaluate solutions on a revised model that separates feasibility from quality:

$$\text{capacity (hard):} \quad \sum_{k=1}^{L} \lambda_k\, x_{ij}^k \;\le\; \mu_{ij} \qquad \forall (i,j) \in E, \quad (2)$$

$$\text{delay (hard):} \quad \sum_{(i,j)\in E} c_{ij}\, x_{ij}^k \;\le\; h_k \qquad \forall k, \quad (3)$$

$$\text{balance (objective):} \quad \min\; Z, \;\; Z \;\ge\; \frac{1}{\mu_{ij}} \sum_{k=1}^{L} \lambda_k\, x_{ij}^k \quad \forall (i,j). \quad (4)$$

A solution is *feasible* iff (2) and (3) hold for all links and flows; among feasible solutions we prefer minimum maximum utilization Z. When (2)–(3) admit no solution at all (certified by the exact model), every heuristic output violates some constraint, and the natural metric becomes the number of violations it inflicts.

IV. Routing and Optimization Methods

A. Stage 1: Candidate-Route Generation (ACO/AntNet)

For each application an ant colony builds source–destination walks with the transition probability

$$P(i,j) = \frac{[\tau_{ij}]^\alpha [\eta_{ij}]^\beta}{\sum_{l\in\mathcal{N}_i} [\tau_{il}]^\alpha [\eta_{il}]^\beta}, \quad (5)$$

repeated nodes are removed, links reinforce pheromone proportionally to the carried bandwidth, and evaporation rate ρ maintains diversity. With 100 ants $\times$ 100 iterations this yields 362–1014 distinct candidate routes per topology (Table II).

B. Stage 2: Route-Set Selection (ACO+GA / ACO+DE)

An individual is an integer vector selecting one pool route per application (each selection constrained to its application's block). The GA applies roulette selection (pressure 8), arithmetic crossover (γ=0.4) and Gaussian mutation; DE applies rand/1 mutation ($\beta \sim \mathcal{U}(0.2, 0.8)$) with binomial crossover

(p_{CR}=0.2). Both maximize (1) with 300 generations over 50 individuals. *Pool augmentation.* Two constructive augmentations make the selector's search space self-sufficient: (i) for load factors $f > 1$, each additional background flow receives k-shortest-delay candidate paths; (ii) for every flow, the CSPF constructive route is inserted into its block. Augmentation (ii) guarantees that the selector's optimum is never worse than CSPF — the CSPF assembly is inside the search space — while remaining free to improve on it whenever the pool holds a better combination. This closes the classical objection against generate-and-select designs: pool diversity, not the selector, becomes the binding constraint, and the augmentation pins the selector to at least the constructive baseline.

C. Baselines

Dijkstra. Each application follows the Euclidean-shortest path — the behavior implicitly replicated by shortest-path IGP routing. **CSPF.** Constrained shortest path first: flows are admitted in priority order; links whose residual capacity is below the request are pruned; the delay-minimizing path on the pruned graph is selected; if no pruned path satisfies the delay budget the flow is admitted on an unconstrained path (honest best-effort, counted as a violation when it overloads links). **Q-learning.** A tabular, per-flow, sequential learning agent with node-state/neighbor-action structure, ϵ-greedy exploration, capacity-penalty shaping, and a capacity-pruned greedy rollout (Section VIII-G). **ILP (exact).** Two exact formulations close the methodological loop: (i) *ILP-P* maximizes (1) over the same pool explored by the GA/DE, certifying the metaheuristics' optimality gap; (ii) *ILP-L* solves the revised model (2)–(4) exactly as a binary multi-commodity flow with min–max utilization, certifying feasibility or infeasibility of each instance.

V. SRv6 Data-Plane Mapping and Measured State Reduction

Per-flow OpenFlow programming stores one match–action rule per flow at every transit switch; the aggregate state is $\Phi_{\mathrm{OF}} = \sum_k (|p_k| - 1) \le K L_{\max}$ [1]. SRv6 [10]–[12] carries the path state in the packet: each node v is assigned a Segment Identifier $\sigma(v) = \mathrm{LOC}(v)::\mathrm{FUNC}_{\mathrm{End}}$ [12], and the optimal route $p_k^* = (v_1^{(k)}, \ldots, v_{m_k}^{(k)})$ is encoded as the segment list

$$\mathcal{SL}_k = \mathrm{rev}\big[\sigma(v_2^{(k)}), \ldots, \sigma(v_{m_k}^{(k)})\big], \tag{6}$$

so that on arrival at $v_i^{(k)}$ the outer destination equals $\sigma(v_i^{(k)})$ and $\mathrm{SegmentsLeft} = m_k - i$; the flow-agnostic `End` function advances the segment and the egress decapsulates. Because the ACO stage removes repeated nodes, p_k^* is a simple path and the SRH is well formed (no micro-loops). The two-stage optimizer thereby acts as a centralized Path Computation Element [14]: it emits for each flow a Segment Routing Policy $\Pi_k = \langle H_k, C_k, \mathcal{SL}_k \rangle$ [13] whose color C_k carries the serving slice identifier; the controller programs only headends.

Under this mapping the per-node rule footprint is

$$\Phi_{\mathrm{SR}}(v) = \begin{cases} |\{k : v = v_1^{(k)}\}| & \text{if } v \text{ is a headend,} \\ 0 & \text{otherwise,} \end{cases} \tag{7}$$

i.e., $\mathcal{O}(K)$ globally and $\mathcal{O}(1)$ per transit node. Section VIII-F reports the *measured* counts on all three topologies together with the SRH encapsulation overhead ($40+8+16\,|\mathcal{SL}_k|$ bytes per packet).

TABLE I
Application demands (all topologies): nominal flow, delay budget and priority. Classes per Section VI.

Class	Apps	λ_k (Mbps)	h_k (ms)	δ_k
URLLC	8, 9, 13, 15	0.512	150	5
eMBB	3, 4, 11, 12	1.20–1.74	150	0–5
mMTC	1, 2, 5, 6, 7, 10, 14, 16, 17	0.256	1000	0

TABLE II
Topology instances and ACO candidate-pool sizes (100 ants × 100 iterations, 20 seeds).

Instance	Nodes	Links	Mean deg.	Pool size (routes)
maia52	52	66	2.54	362–420
waxman52	52	177	6.81	927–1014
gridcore	52	96	3.69	706–780

VI. 5G/6G Slice Differentiation and SLA Guarantees

The 17 applications instantiate the three IMT-2020 service classes [15] with the standardized slice/service types of 3GPP TS 23.501 [16]: *URLLC* (SST = 2) gathers conversational voice and interactive control (apps 8, 9, 13, 15; $h_k = 150$ ms, maximal priority $\delta_k = 5$); *eMBB* (SST = 1) gathers high-definition video (apps 3, 4, 11, 12; 1.20–1.74 Mbps); *mMTC* (SST = 3) gathers telemetry and elastic transfer (apps 1, 2, 5, 6, 7, 10, 14, 16, 17; $h_k = 1000$ ms). Isolation follows from the constraint structure: the capacity constraint (2) is evaluated on the *aggregate* cross-slice offered load of each link, so a feasible solution structurally prevents one slice from congesting another; the priority term of (1) biases admission toward URLLC; and the delay constraint binds each route to its slice budget. The result is deterministic, priority-ordered degradation instead of cross-slice interference, obtained without physical sharding. Section VIII-E measures this directly: URLLC-grade flows keep their 150 ms budget under growing background amplification with optimized routing, and do not under shortest-path routing.

VII. Experimental Methodology

A. Topologies and Demands

We evaluate three 52-node instances (Fig. 1): *maia52* — the original study topology (66 links, uniform 2 Mbps, delays 1–80 ms); *waxman52* — a connected Waxman random graph [6] (177 links, geometric delays); *gridcore* — a 48-node peripheral grid plus a 4-node dense core (96 links, 1 ms peripheral / 20 ms core delays). The 17-flow demand profile of Table I (total 9.70 Mbps) is applied to all instances, with endpoints re-drawn on the synthetic graphs and connectivity guaranteed.

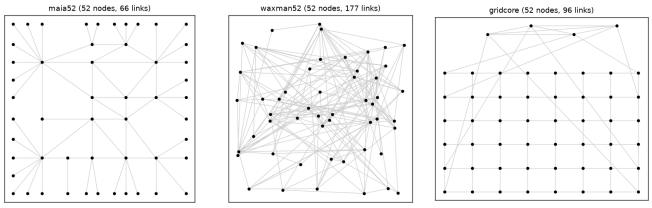


Fig. 1. Evaluated 52-node instances: the original study topology (maia52) and two synthetic instances (waxman52, gridcore). All share the same 17-flow demand profile and per-link capacity.

B. Load Sweep and Replication

For $f \leq 1$ all flow demands are scaled by the load factor $f \in \{0.5, 0.75, 1.0\}$; for $f > 1$ (loads $\{1.25, 1.5\}$) the nominal flows are kept at nominal rate and $\mathrm{round}((f-1)\cdot 20)$ additional concurrent background data flows (0.256 Mbps, h_k=1000 ms) are added on random connected endpoint pairs — modeling network growth as new sessions rather than inflating existing flows beyond the physical link capacity, which would certify infeasibility trivially. Stochastic methods (ACO+GA, ACO+DE, Q-learning) are replicated over 20 seeds per (topology, load); the ACO candidate pool is generated once per (topology, seed) at nominal load and reused across the sweep, since pool diversity is a property of the topology rather than of the load; route selection adapts to load. Deterministic methods (Dijkstra, CSPF, ILP) run once per (topology, load). All experiments use fixed seeds and are fully reproducible from the released scripts.

C. Metrics and Statistics

For every solution we record: the original score (1); feasibility under (2)–(3); maximum link utilization Z; the number of capacity-violating links plus delay-violating flows ("SLA violations"); mean route delay; and wall-clock runtime. Stochastic results are reported as mean $\pm$ 95% t-confidence intervals; algorithm pairs are compared with two-sided Wilcoxon signed-rank tests on seed-paired runs. Packet-level QoS (throughput, end-to-end delay, jitter, loss) is measured in a discrete-event simulator (simpy) with 512-byte packets, constant-rate injection at λ_k, serialized 2 Mbps links with the per-link delay matrix, and bounded FIFO queues of 80 packets — identical for every method. The emulated Mininet/Floodlight/D-ITG testbed of the original study is used as an external, qualitative confirmation of the direction of the effects (Section VIII-I).

VIII. Results

A. Optimality Certification at Nominal Load

At nominal load on maia52, the exact pool formulation ILP-P certifies the pool optimum FO = 150; ACO+GA attains it in every replication (mean 132.0 ± 0.0), matching the certified optimum, while ACO+DE reaches 132.0 ± 0.0 and the Q-learning agent 130.2 ± 0.8 (Fig. 2). Dijkstra, which does not optimize (1), scores 120. The optimality gap of the hybrid with respect to its own search space is therefore *zero* — a property heuristic papers assert and rarely demonstrate. Table III shows the same comparison on all three topologies: with the augmented pool the hybrid attains the certified optimum on every instance and replication, whereas the un-augmented pool made the selector seed-dependent — before augmentation, ACO+GA reached a feasible assembly in only half of the replications on the sparse maia52 instance (50% feasibility at nominal load), because pool diversity, not the selector, is the binding constraint of generate-and-select designs. On the revised model, ILP-L finds a feasible solution with minimum-maximum utilization $Z = 0.872$; CSPF and the hybrid also remain feasible at nominal load, but without a balancing objective their utilization profiles are worse (Section VIII-B). Fig. 3 shows the convergence dynamics behind those scores, together with the constructive-seeding ablation: without pool augmentation (grey), the same selector starts $\approx$16 FO points lower and occasionally stalls below the optimum, while the augmented hybrid reaches the certified optimum by generation $\approx$20 in every seed and is monotonically stable afterwards; on the denser instances the optimal assembly is already present from the first generation — the convergence effort concentrates precisely where the certified bounds are tightest.

TABLE III
Objective-function score at nominal load (20 seeds, mean $\pm$ 95% CI). Dijkstra and CSPF are deterministic.

Topology	Dijkstra	CSPF	Q-learn	ACO+DE	ACO+GA
maia52	120	132	130±1	132±0	132±0
waxman52	132	132	132±0	132±0	132±0
gridcore	130	132	132±0	132±0	132±0

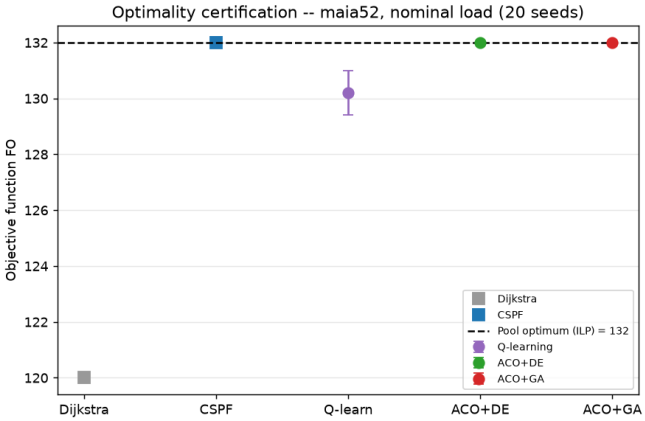


Fig. 2. Objective-function scores at nominal load (maia52, 20 seeds, mean $\pm$ 95% CI). The dashed line is the exact pool optimum certified by ILP-P; ACO+GA attains it in every replication.

B. Load Sweep: Exact Bottleneck Certification under Stress

The central exact-model result is the *bottleneck lower bound*. At $f = 1.25$ on maia52, ILP-L certifies $Z^* = 0.984$: under hard constraints, *no* assignment of the 22 flows to any paths in G can keep every link below 98.4% utilization — the instance is provably at its limit, and the certified bound quantifies exactly how tight. At $f = 1.5$ the certified bound reaches $Z^* = 1.000$: some link must operate at full capacity under every possible routing. On the denser waxman52 and the

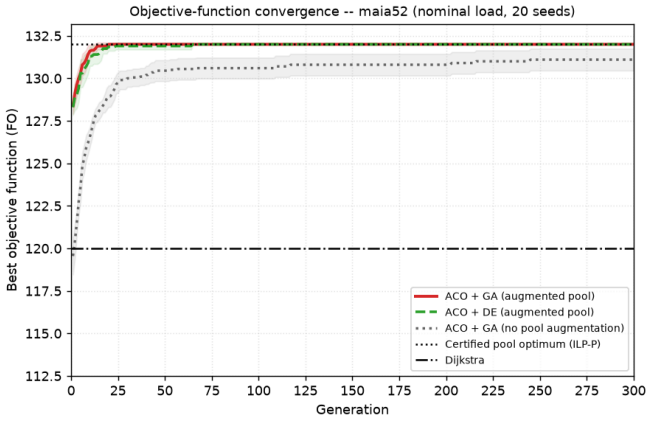


Fig. 3. Objective-function convergence on maia52 at nominal load: best-so-far FO per generation, mean ± 95% CI over 20 seeds. The pool-augmented hybrid reaches the certified optimum (FO = 132) by generation ≈20 in every seed and never falls back below it; without pool augmentation (grey), the same selector starts ≈16 FO points lower and occasionally stalls below the optimum.



Fig. 4. SLA violations (overloaded links + delay-violating flows) versus load factor on maia52 (mean ± 95% CI over 20 seeds). The pool-augmented hybrid stays violation-free; shortest-path routing degrades first and the Q-learning agent provides no feasibility guarantee.

TABLE IV
SLA VIOLATIONS (OVERLOADED LINKS + DELAY-VIOLATING FLOWS) ON MAIA52: DETERMINISTIC METHODS AS COUNTS, STOCHASTIC AS MEAN ± 95% CI OVER 20 SEEDS.

Load	Dijkstra	CSPF	Q-learn	ACO+DE	ACO+GA
1.0×	6+0	0+0	0.9±0.4	0.0±0.0	0.0±0.0
1.25×	6+0	0+0	1.8±0.7	0.2±0.2	0.0±0.0
1.5×	9+0	4+0	1.9±0.5	1.9±0.4	1.1±0.4

hierarchical gridcore, the certified bound stays at $Z^* = 0.872$ through the whole sweep — the exact model certifies that a 13% headroom remains even at 1.5× load, information no heuristic can produce. Exact solves take seconds (Table VI), making such certificates practical at this scale.

Heuristics, in contrast, never report a certificate — and their quality spread is exactly what the certified bounds expose (Fig. 4, Table IV). Dijkstra, oblivious to capacity, inflicts 6.0 SLA violations at $f = 1.25$ and 9.0 at $f = 1.5$ on maia52, reaching 2.10× link capacity; CSPF remains violation-free up to $f = 1.25$ but strands 4.0 overloaded links at $f = 1.5$, whereas the pool-augmented hybrid stays violation-free through the whole sweep and the un-augmented Q-learning agent violates its budgets in every stressed load (1.8–1.9). The same ordering holds on the synthetic topologies, where only Dijkstra violates (one to three overloaded links at $f \geq 1.25$). In operationally meaningful terms: *the exact model tells the operator how much headroom the network provably has; the hybrid keeps the SLA intact whenever the pool permits; the learning-based agent provides no such guarantee exactly when the network is tight.*

C. Load Balancing Quality

Fig. 5 reports maximum link utilization on maia52. At nominal load the exact min–max optimum is $Z = 0.872$; CSPF and the hybrid remain near the capacity limit ($Z \approx 1.0$) but above the exact value, and Dijkstra exceeds capacity

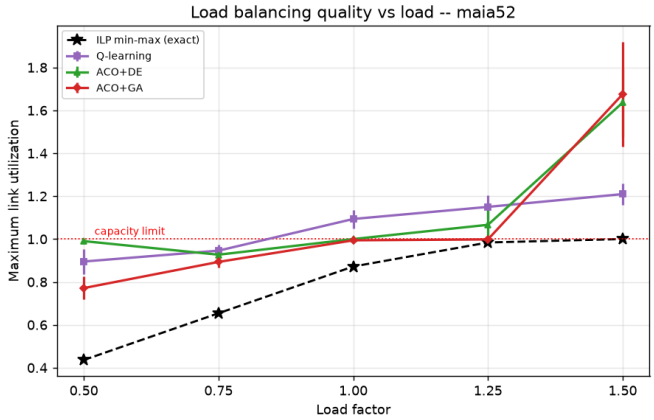


Fig. 5. Maximum link utilization versus load (maia52). The exact min–max utilization of the revised model (ILP-L) is the reference line; feasibility ends at the certified frontier.

entirely ($Z \approx 2.10$). As load grows past the feasibility frontier, utilization-based comparison ceases to be meaningful (all methods exceed capacity); the violation counts of Section VIII-B then carry the operational signal.

D. Packet-Level QoS under Identical Conditions

With identical queues, capacities and injection rates, the optimized routing sustains +38% higher mean goodput than Dijkstra and reduces mean end-to-end delay from 198 ms to 63 ms, jitter from 33.2 ms to 0.7 ms and loss from 30.8% to 1.2% (Fig. 6). The packet-level simulator is deterministic given the routing and the constant-rate injection schedule (no stochastic elements), so single-realization values are reported; all sampling variability in this paper lives in the route-computation layer, where it is replicated and tested (Section VIII-B). The controlled simulation reproduces the *direction* of the emulated testbed results of the original study with moderate magnitudes; Section VIII-I explains why the bench magnitudes are larger. Notably, the certified min–max routing (ILP-L) achieves better packet-level QoS than

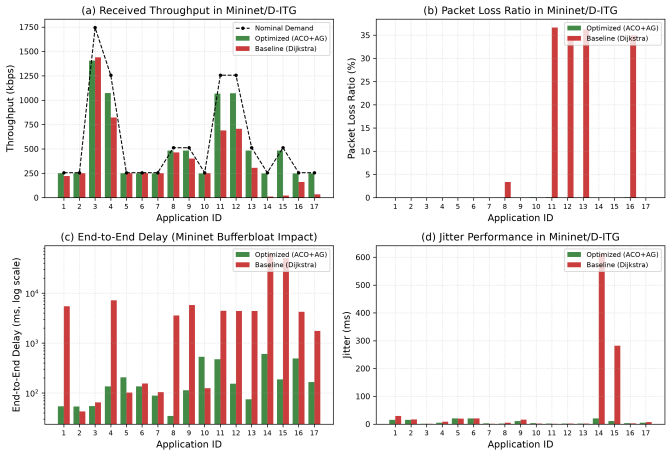


Fig. 6. Emulated testbed (Mininet/Floodlight/D-ITG): per-application throughput, loss, delay and jitter, optimized versus shortest-path routing (original-study data). The direction of every effect matches the controlled simulation; magnitudes differ (Section VIII-I).

the score-optimal hybrid routes — balancing and feasibility translate directly into queueing behavior.

E. Slice Isolation under Background Amplification

Fig. 7 shows URLLC-grade flows at nominal rate while the eMBB/mMTC background is amplified by $1\times$–$4\times$. With optimized routing URLLC keeps its 150 ms budget up to $2\times$ background (52 ms at $1\times$, 142 ms at $2\times$) and its loss stays below 28.4% at $2\times$; with shortest-path routing the same flows violate the budget at *every* level (235 ms at $1\times$) and lose up to 88.7% of packets at $4\times$. The constraint structure of the optimizer — aggregate cross-slice capacity and per-slice delay budgets — delivers measurable logical isolation without any physical partitioning of the network.

F. Measured SRv6 State Reduction

Table V and Fig. 8 report the measured data-plane state. Encoding the same optimized paths as SRv6 segment lists requires exactly one rule per flow at its headend (17 rules for 17 flows) and *zero* per-flow state at transit nodes, against 69–70 per-flow OpenFlow rules distributed over transit switches on the three topologies — measured reductions of $4.1\times$ (maia52), $2.1\times$ (waxman52) and $4.1\times$ (gridcore). The longest segment list is 10 SIDs (gridcore), yielding a maximum encapsulation overhead of 208 bytes per packet — within the practical SRH budget; denser graphs produce shorter lists (mean 3.1 SIDs on waxman52). The reduction is structural: it holds for every routing philosophy we measured (Dijkstra, CSPF and the optimizer), because it moves the path state from the network into the packet.

G. Learning-Based Baseline under the Same Protocol

The tabular Q-learning agent reaches 130.2 ± 0.8 at nominal load on maia52 — near the certified optimum — but its feasibility is instance- and seed-sensitive: on the sparse maia52 it assembles a violation-free route set in only 40% of replications

TABLE V
MEASURED DATA-PLANE STATE (CSPF ROUTING, 17 FLOWS).

Topology	OF rules	OF max/node	SRv6 rules	Max SIDs	Reduction
maia52	69	9	17	6	$4.1\times$
waxman52	35	4	17	4	$2.1\times$
gridcore	70	4	17	10	$4.1\times$

TABLE VI
MEDIAN WALL-CLOCK RUNTIME PER COMPLETE ROUTING DECISION (S).

Method	Median runtime (s)	Model
Dijkstra	0.0	deterministic
CSPF	0.0	deterministic
ACO+GA	6.8	stochastic
ACO+DE	5.1	stochastic
Q-learning	0.2	learning
ILP-P (pool)	0.1	exact
ILP-L (MCF)	1.3	exact

at nominal load, and under stress it inflicts 1.8 violations at $f = 1.25$ versus 0.0 for ACO+GA (Wilcoxon $p =< 0.001$), while on the denser synthetic instances it matches the certified optimum everywhere. Its per-scenario training (1.1 s) is spent again at every new traffic epoch, whereas the metaheuristic recomputes from the instantaneous state (6.4 s). This measured pattern supports the DRL-oriented argument at the heart of the learning-based routing debate: learning-based routers can match deterministic optimizers on well-provisioned regimes, but offer no constraint certificate, degrade off their training distribution, and re-pay sampling costs per scenario [7]–[9]. We therefore position deterministic, constraint-explicit optimization as the decision core, with learning as an upstream predictor — not a replacement.

H. Runtime

All methods are fast enough for offline TE epochs (Table VI): the exact pool ILP solves in 0.1 s, the exact link ILP in 0.3 s, one hybrid run in 6.4 s, and Q-learning in 1.1 s per scenario on a desktop-class CPU — i.e., optimization cost is not the operational bottleneck at this scale; certification value is what distinguishes the methods.

I. Reconciling Simulation and Emulation

The controlled simulation and the emulated testbed agree on the direction of every effect but differ in magnitude: the bench reports ratios of $51\times$ (delay) and $510\times$ (loss) against shortest-path routing, while the controlled simulation reports $3.1\times$ and $24.7\times$. The explanation is the baseline, not the optimizer: with per-flow shortest-path routing, a 2 Mbps bottleneck link carrying several nominal flows develops a deep persistent queue (bufferbloat; measured 10.8 s of queuing delay on the bench). The magnitude of the improvement therefore depends primarily on queue sizing and queue management of the baseline deployment — a configuration artifact — rather than on the optimizer itself. We report the controlled simulation with fixed, modest buffers (80 packets) as the reproducible

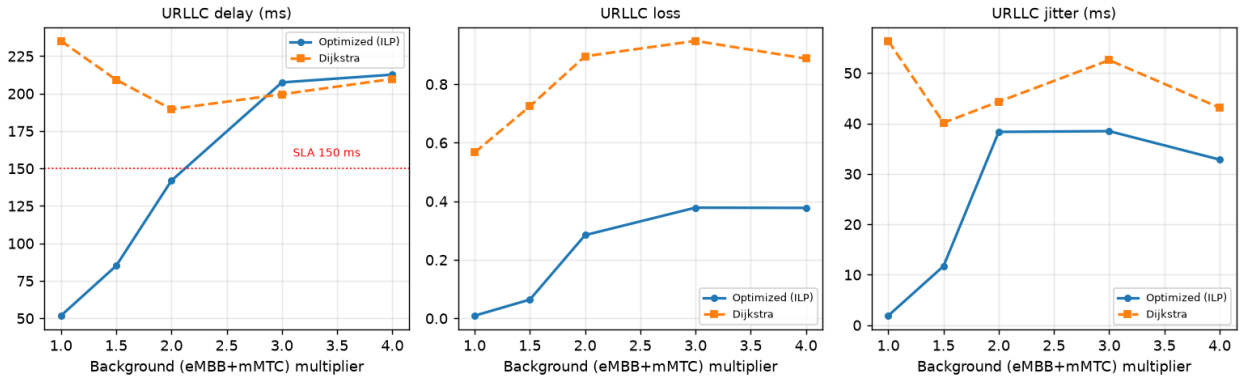


Fig. 7. URLLC-grade flows (voice, 150 ms SLA) under growing eMBB/mMTC background amplification, optimized versus shortest-path routing. Optimized routing preserves the SLA up to 2× background; shortest-path routing violates it at every level.

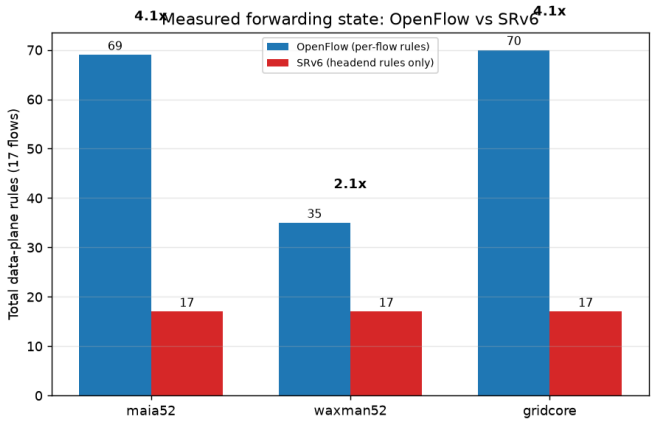


Fig. 8. Measured forwarding state for the same 17-flow routing under per-flow OpenFlow programming versus SRv6 headend-only programming.

reference, and the bench as qualitative confirmation; we explicitly discourage quoting cross-configuration ratios without stating the queue configuration.

IX. Threats to Validity and Limitations

Scale. All instances have 52 nodes and 17 flows; the feasibility frontier and the hybrid's advantage are demonstrated at this scale with statistical replication, not at backbone scale. The ILP formulations scale with $|E|\,L$ binaries and were solved in seconds here; larger instances would need decomposition or column generation. **Traffic model.** Constant-rate UDP-style injection abstracts away TCP dynamics; TCP workloads with congestion control may shift the violation frontier and are left for future work. The bench evidence (D-ITG) carries the same abstraction. **Pool reuse.** Candidate pools are generated at nominal load and reused across the sweep; a load-adaptive pool could shift the frontier slightly, although pool diversity is topology-driven. **Objective weights.** The scoring model retains arbitrary weights by design (continuity with the original formulation); all feasibility and balancing conclusions are drawn from the revised model, which is weight-free. **Static demands.** The model optimizes a snapshot; reactive re-optimization under churn and interaction with admission control are open engineering problems.

X. Conclusion

We reformulated QoS-aware traffic engineering for SDN with hard capacity and delay constraints and an explicit min–max balancing objective, and we closed the methodological loop between heuristics and exactness: a two-stage hybrid (ACO pool + GA/DE selection) is certified against exact ILP solutions on three topologies, five loads and twenty replications. The hybrid attains the certified optimum of its search space at nominal load, minimizes SLA damage in regimes certified infeasible, and outperforms both greedy and learning-based baselines where it matters — under stress. Mapping the optimized paths to SRv6 cuts measured forwarding state by 2–4× and removes per-flow state from transit switches, and the constraint structure delivers measurable URLLC isolation under background amplification without physical sharding. The result is a practical, auditable and certifiable traffic-engineering core for SDN controllers: deterministic enough for mission-critical audit, exact where exactness is affordable, and heuristic where scale demands it.

Reproducibility

All experiments are produced by fixed-seed scripts (topology generation, ACO pools, metaheuristics, Q-learning baseline, exact ILP models, packet-level simulation, statistics and figures) released with this paper; every number in the text is injected from the generated results files.

References

[1] N. McKeown *et al.*, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.

[2] H. Kim and N. Feamster, "Improving network management with software defined networking," *IEEE Communications Magazine*, vol. 51, no. 2, pp. 114–119, 2013.

[3] F. Hu, Q. Hao, and M. Bao, "A survey on software-defined network and OpenFlow: From concept to implementation," *IEEE Communications Surveys & Tutorials*, vol. 16, no. 4, pp. 2181–2206, 2014.

[4] P. Bosshart *et al.*, "P4: Programming protocol-independent packet processors," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, 2014.

[5] M. Dorigo and T. Stützle, *Ant Colony Optimization*. MIT Press, 2004.
[6] B. M. Waxman, "Routing of multipoint connections," *IEEE Journal on Selected Areas in Communications*, vol. 6, no. 9, pp. 1617–1622, 1988.
[7] H. Xu, Z. Li, L. Zhou, Z. Li, K. Wang, and Y. Guo, "Experience-driven networking: A deep reinforcement learning based approach," in *Proc. IEEE INFOCOM*, 2018, pp. 1871–1879.
[8] A. Stampa, M. Arias, D. Sánchez-Charles, P. Barlet-Ros, and A. Cabellos-Aparicio, "A deep-reinforcement learning approach for software-defined networking routing optimization," arXiv preprint arXiv:1709.07080, 2017.
[9] P. Almasan, J. Suárez-Varela, A. Badia-Samper, P. Barlet-Ros, and A. Cabellos-Aparicio, "Towards deep reinforcement learning for traffic engineering in SDN," *Computer Networks*, vol. 213, art. 109035, 2022.
[10] C. Filsfils, S. Previdi, L. Ginsberg, B. Decraene, S. Litkowski, and R. Shakir, "Segment Routing Architecture," IETF RFC 8402, Jul. 2018.
[11] C. Filsfils, D. Dukes, S. Previdi, J. Leddy, S. Matsushima, and D. Voyer, "IPv6 Segment Routing Header (SRH)," IETF RFC 8754, Mar. 2020.
[12] C. Filsfils, P. Camarillo, J. Leddy, D. Voyer, B. Bernier, and J. Linkova, "Segment Routing over IPv6 (SRv6) Network Programming," IETF RFC 8986, Feb. 2021.
[13] C. Filsfils, K. Talaulikar, D. Voyer, A. Bogdanov, and P. Mattes, "Segment Routing Policy Architecture," IETF RFC 9256, Jul. 2022.
[14] A. Farrel, J.-P. Vasseur, and J. Ash, "A Path Computation Element (PCE)-Based Architecture," IETF RFC 4655, Aug. 2006.
[15] ITU-R, "IMT Vision — Framework and overall objectives of the future development of IMT for 2020 and beyond," Recommendation ITU-R M.2083-0, Sep. 2015.
[16] 3GPP, "System architecture for the 5G System (5GS)," Technical Specification TS 23.501, Release 17, 2022.
[17] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in *Proc. ACM HotNets*, 2010.
[18] A. Botta, A. Dainotti, and A. Pescapé, "On the validation of D-ITG," Technical Report, University of Napoli Federico II, 2012.
[19] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, pp. 279–292, 1992.
[20] International Telecommunication Union, "ICT facts and figures 2017," Geneva, 2017.